CONFLUENT CLOUD

Architecture • Deployment • Stream Processing • AI Integration

Prepared by: Rajesh Pasham

Target: IT Professionals | DBAs | Integration Engineers | Cloud Architects

Course Agenda

13 Comprehensive Topics Covering Confluent Cloud End-to-End

01 Introduction to Confluent Cloud

02 Cluster Types & Deployment Options

03 Architecture & Core Components

04 Pricing Models & Cost Management

05 Security Features & Compliance

06 Data Integration: Connectors & Kafka Connect

07 Stream Processing: ksqlDB & Apache Flink

08 Schema Management & Data Governance

09 Monitoring, Observability & Operations

10 Comparison: Confluent vs. Alternatives

11 Migration Strategies & Best Practices

12 AI & Real-Time Intelligence

13 Case Studies & Real-World Applications

Section 1: Introduction to Confluent Cloud

Cloud-Native, Fully Managed Apache Kafka Platform

What is Confluent Cloud?

- Cloud-native, fully managed Apache Kafka platform (PaaS)
- Available across AWS, Microsoft Azure, and Google Cloud
- Designed for elasticity, multi-tenancy & operational excellence
- Not just hosted Kafka — built for the cloud from the ground up

Ideal Use Cases

- Reduce operational burden (no Kafka expertise needed)
- Rapid deployment: days vs. months for self-managed
- Multi-cloud flexibility across AWS, Azure, GCP
- Variable or unpredictable workloads (elastic scaling)
- Strict compliance/security requirements

Why Confluent Cloud vs Alternatives?

- Hosted Kafka: You manage all operations yourself
- AWS MSK: AWS manages brokers; you manage topology
- Confluent Cloud: Fully managed + auto-scaling + ecosystem
- ————————
- Speed: Provision in minutes, not weeks
- Cost: Pay-per-use, scale-to-zero, no over-provisioning
- SLA: 99.99% uptime (Standard & Enterprise tiers)
- 120+ fully managed connectors — no infra needed
- Integrated Schema Registry, ksqlDB, Apache Flink
- Multi-cloud deployment with consistent API parity

Section 2: Cluster Types & Deployment Options

Five Distinct Cluster Types for Every Need

	Basic	Standard	Enterprise	Dedicated	Freight
STARTING PRICE	$0–$0.14/eCKU-hr	$0.75/eCKU-hr	$1.75–$2.25/eCKU-hr	Custom	$2.25/eCKU-hr
SLA	99.5%	99.99%	99.99%	99.95–99.99%	99.99%
BEST FOR	Dev / Test / POC	Production	Mission-Critical	Ultra-High Scale	Logging / Observability
	No Private Net	No Private Net	Private Network	Private Network	No Private Net
	Auto-Scaling	Auto-Scaling	Auto-Scaling	Manual Scale	Auto-Scaling

Section 3: Confluent Cloud Architecture & Components

A Complete Data Streaming Ecosystem

Schema Registry

Schema versioning & validation

Kafka Brokers

Core messaging backbone

Kafka Connect

80+ Managed Connectors

ksqlDB

SQL stream processing

Apache Flink

Advanced stream & batch

Cluster Linking

Cross-cluster replication

Schema Registry Details

- Central repo for Avro, Protobuf, JSON Schema
- Enforces backward / forward / full compatibility
- Auto-integrated with Kafka clients & ksqlDB
- Included at no extra cost with Confluent Cloud

Cluster Linking

- Mirror topics across clusters with zero data loss
- Automatic consumer offset mapping during migration
- Supports Disaster Recovery, DR, and Multi-Region
- Works with PrivateLink and VPC Peering

Section 4: Pricing Models & Cost Management

Consumption-Based Pricing — Pay for What You Use

Compute (eCKU)

- Basic: $0.14/eCKU-hour
- Standard: $0.75/eCKU-hour
- Enterprise: $1.75–$2.25/eCKU-hour
- Scale-to-zero: $0 when cluster is idle

Storage

- Standard/Enterprise: $0.08/GB-month
- Freight: $0.03/GB-month (10x cheaper)
- Charged for actual data retained
- Optimize by tuning retention periods

Data Transfer

- Ingress: $0.05/GB
- Egress Standard: $0.035–$0.05/GB
- Freight egress: $0.014–$0.03/GB
- Cross-cloud billed by cloud provider

Estimated Monthly Costs by Scenario

Scenario	Cluster	Throughput	Est. Monthly Cost
Dev / Testing	Basic	1 MB/s	~$0 (free tier)
Small Production	Standard	50 MB/s	~$1,642+
Large Production	Enterprise	500 MB/s	~$26K+
Logging / Observability	Freight	1 TB/day	~$300/month storage

Section 5: Security Features & Compliance

Enterprise-Grade Security Built In

Encryption

- TLS 1.2+ for all data in transit
- AES-256 encryption at rest by default
- BYOK (Bring Your Own Key) on Enterprise/Dedicated
- Customer-managed keys via AWS KMS / Azure Key Vault

Authentication & Authorization

- API Keys (service-to-service bearer tokens)
- OAuth 2.0 / OIDC for SSO integration
- SAML 2.0 for federated identity
- RBAC: Granular roles (Admin, Operator, Developer)

Network Security

- VPC/VNet Peering (Enterprise+)
- AWS PrivateLink & Azure Private Link
- AWS Transit Gateway support
- IP allowlisting for additional control

Compliance Certifications

- SOC 2 Type II — Security & availability
- SOC 3 — Public-facing SOC 2 summary
- ISO 27001 — Information security standard
- PCI DSS — Payment card industry compliance
- HIPAA Business Associate Agreement available
- GDPR — Data processor with DPA support

Audit & Governance

- Audit logs: All actions logged (login, access, config)
- Sent to dedicated audit topic or external SIEM
- Stream Governance: Data catalog & lineage tracking
- Schema-level access controls via RBAC
- Field-level masking for sensitive PII data

Section 6: Data Integration — Kafka Connect

80+ Fully Managed Connectors | Zero Infrastructure Overhead

How Kafka Connect Works

- Source Connectors: Pull data from external systems → Kafka
- Sink Connectors: Push data from Kafka → external systems
- Single Message Transforms (SMT): Filter/mask/map in-flight
- Confluent hosts & scales workers — no infra management

Fully Managed vs. Custom Connectors

- Fully Managed (80+): Zero infra overhead; Confluent operates
- Custom Connectors: You provide code; Confluent Cloud hosts
- Cost: ~$0.05/connector task/hour for managed connectors
- Schema Registry integration for automatic validation

Databases

MySQL, PostgreSQL, Oracle, SQL Server → CDC

Data Warehouses

Snowflake, BigQuery, Redshift → Real-time load

Cloud Storage

S3, Azure Blob, GCS → Data lake / archival

SaaS & APIs

Salesforce, ServiceNow, HTTP, REST Webhooks

Search & Analytics

Elasticsearch, MongoDB, Splunk

Section 7: Stream Processing — ksqlDB & Apache Flink

Real-Time Transformations, Aggregations & AI Inference

ksqlDB — SQL for Kafka Streams

- SQL-like syntax — lower learning curve than Kafka Streams
- Stateful aggregations: COUNT, SUM, AVG over time windows
- Stream joins: Enrich events with lookup data
- Windowing: Tumbling, hopping, session windows
- Push queries: Continuous result streams (EMIT CHANGES)
- Cost: ~$0.10/CSU-hour (Confluent Streaming Unit)

Apache Flink — Advanced Stream Processing

- More powerful than ksqlDB for complex algorithms
- Supports Java, Python, Scala — greater flexibility
- Native ML Inference: Run models inside Flink SQL
- Flink Search: Vector similarity vs. MongoDB, Pinecone
- Snapshot Queries (2025): Combine batch + real-time data
- Cost: ~$0.09–$0.12/CFU-hour

Example: Real-Time Order Aggregation

```
CREATE TABLE order_counts AS
SELECT customer_id,
  COUNT(*) AS orders,
  SUM(amount) AS total
FROM orders
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY customer_id;
```

Example: Flink Native ML Inference

```
CREATE TABLE fraud_scores AS
SELECT transaction_id, amount,
  MODEL 'fraud_model_v3'
  INPUTS (txn_id, amount, history)
  OUTPUTS (fraud_score, risk_level)
FROM transactions
WHERE fraud_score > 0.7;
```

Section 8: Schema Management & Data Governance

Enforce Data Contracts Across All Producers & Consumers

Schema Registry Core Concepts

- Producer registers schema → receives Schema ID
- Messages carry Schema ID (not full schema) for efficiency
- Consumer fetches schema by ID → deserializes correctly
- Supports Avro (binary), Protobuf, JSON Schema formats

Schema Compatibility Modes

Mode	Can Read	Best For
Backward	Old data with new schema	Producer upgrades
Forward	New data with old schema	Consumer upgrades
Full	Both directions	Most flexibility
None	No checks	Development only

Stream Governance

- Data Catalog: Auto-discover and tag all topics/schemas
- Data Lineage: Track data flow from source to consumer
- Quality Rules: Auto-validate fields (non-null, regex, range)
- Business metadata: Tag topics with owner, team, SLA
- Available in Essential (free) and Advanced tiers

Schema Evolution Best Practices

- Always add new fields with default values (backward compat)
- Never rename or remove required fields in production
- Use Full compatibility for multi-team schemas
- Test schema changes in dev cluster before promoting
- Use CI/CD pipeline to validate schema changes pre-deploy

Section 9: Monitoring, Observability & Operations

Built-In Intelligence — No Custom Tooling Required

Throughput Metrics

- eCKU utilization (current/hourly/daily)
- Messages in/out per second
- Bytes in/out per second
- Peak vs. baseline throughput

Consumer Health

- Consumer lag (unconsumed messages)
- Rebalancing events frequency
- Failed offset commits
- Consumer group status

Data Quality

- Schema validation error rates
- Null rates in key fields
- Connector error counts
- Dead letter queue (DLQ) volume

Cost Monitoring

- eCKU usage (real-time estimates)
- Storage by topic/consumer group
- Data transfer (ingress/egress)
- Monthly cost trend vs. prior month

Section 10: Confluent Cloud vs. Alternatives

Objective Comparison to Help You Choose the Right Solution

Dimension	Confluent Cloud	AWS MSK	Self-Managed
Setup Time	Minutes	Hours	Days/Weeks
Schema Registry	Included	Separate (Glue)	Manual setup
ksqlDB	Included	Not available	Not available
Stream Processing	Flink (managed)	Not native	Kafka Streams (DIY)
Multi-cloud	AWS + Azure + GCP	AWS only	Any (DIY)
Pricing Model	Usage-based / elastic	Broker-hour based	Infra + staff
Ops Overhead	Minimal	Medium	Very High (50%+ FTE)
Cost (Dev/Test)	Free tier	Min broker cost	$5K+/month
Compliance	SOC2, ISO, HIPAA	AWS shared model	You validate

Section 11: Migration Strategies & Best Practices

Move to Confluent Cloud with Zero Downtime

Recommended: Parallel Cluster Approach

- Phase 1 (2-4 weeks): Run old + new clusters in parallel via Cluster Linking
- Phase 2: Switch consumers to Confluent Cloud; old cluster = read-only fallback
- Phase 3: Validate 2 weeks, then decommission old cluster
- Automatic offset mapping ensures consumers resume correctly

Alternative: Staged Consumer Migration

- Week 1: Migrate non-critical consumers (analytics, logging)
- Week 2: Migrate critical consumers (transactions, notifications)
- Week 3-4: Migrate producers → decommission old infrastructure
- Rollback: Old cluster serves read-only at each stage

Common Migration Pitfalls

- ACLs/RBAC not migrated → apps cannot access topics
- Consumer offsets not mapped → reprocesses all data from start
- Schema versions not exported → deserialization failures
- Fix: Use Cluster Linking for offset sync + schema export/import

Pre-Migration Checklist

- Inventory all topics: count, size, retention periods
- Document all producers and consumer applications
- List compliance requirements (encryption, retention, audit)
- Set up Cluster Linking before any consumer migrations
- Validate data consistency with random topic sampling

Section 12: AI & Real-Time Intelligence

Confluent Cloud Powers the AI Data Pipeline

Flink Native Inference

- Run pre-trained ML models inside Flink SQL
- Inference latency: <10ms (no network hops)
- Data never leaves Confluent Cloud (secure)
- No external ML API calls = cost savings
- _____
- Use Cases:
- Real-time fraud scoring
- Predictive maintenance
- Customer churn prediction
- Dynamic pricing models

Vector DB Enrichment (Flink Search)

- Semantic similarity search on streaming data
- Connect to MongoDB Atlas, Pinecone, etc.
- Enrich events with nearest-neighbor results
- _____
- Use Cases:
- Real-time product recommendations
- Semantic news/content matching
- Customer 360 enrichment
- Similar transaction clustering

AI Streaming Agents (2025)

- Autonomous AI agents triggered by stream events
- Combine Flink + LLM inference + vector search
- Full pipeline: detect → analyze → decide → act
- _____
- Example Agent:
- Shipment event detected
- Query vector DB for past delays
- Predict delay (ML model)
- Auto-notify customer + update ETA
- All in <100ms, fully autonomous

Section 13: Real-World Case Studies

Proven ROI Across Industries

Retail: Omnichannel Inventory Sync

Challenge: 1-hour inventory lag → stock-out and mis-matched availability

Solution: ksqlDB aggregates POS + web + mobile + warehouse streams

Results: Sync: 1hr → 5sec | Stock-outs -15% | Sales +22%

Cost/ROI: $12K/month | ROI: 3 months

Finance: Real-Time Fraud Detection

Challenge: Overnight batch fraud detection → 24-hour latency, $2M+ loss/year

Solution: Flink SQL + ML models: velocity, geo, anomaly, device checks

Results: Detection: 24hr → <1sec | Fraud -68% | $1.4M savings/yr

Cost/ROI: $200K Confluent cost vs $1.4M fraud prevented

Logistics / IoT: Supply Chain Visibility

Challenge: 100K+ shipments, polling-based tracking, no delay prediction

Solution: Freight cluster ingests 500K IoT events/sec (GPS, temp, vibration)

Results: Delay forecast: +72hr window | Fuel savings: 12% | Damage -40%

Cost/ROI: $18K/month vs $200K+ self-hosted

Media: Real-Time Personalization

Challenge: Daily batch recommendations → stale content, 2.1% CTR

Solution: ksqlDB: watch history + co-viewing + trends + ML CTR model

Results: Latency: daily → <100ms | CTR: 2.1% → 8.7% | Revenue +$45M/yr

Cost/ROI: $8K/month | ROI: <1 month

Hands-On Exercise 1: Provision Your First Confluent Cloud Cluster

Topic Area: Cluster Setup | Author: Rajesh Pasham

Objective: Create a free Confluent Cloud account, provision a Basic cluster, and explore the UI.

Step-by-Step Instructions

1. Go to confluent.cloud → Sign Up (use email or Google/GitHub SSO)
2. After login, click 'Add cluster' → Select 'Basic' tier
3. Choose cloud provider (AWS) and region (us-east-1)
4. Name your cluster: my-first-cluster → Click 'Launch cluster'
5. Explore the UI: Topics, Schema Registry, Connectors tabs
6. Create your first topic: Navigate to Topics → 'Add topic'
7. Name it 'test-events', set partitions = 3, retention = 1 day
8. Navigate to Cluster Overview → verify cluster status is 'Running'

Prerequisites: Confluent Cloud account (free tier), modern web browser

Validation Criteria

- Cluster shows status: Running
- Topic 'test-events' created with 3 partitions
- Can view cluster metrics on Overview dashboard
- Schema Registry endpoint is visible in cluster settings

Pro Tips

- Free tier includes 1 eCKU free — no credit card needed
- Record the Bootstrap Server URL for later exercises
- Enable Schema Registry before creating topics in production
- Set meaningful retention based on your use case before going live

Hands-On Exercise 2: Produce & Consume Messages via Confluent CLI

Topic Area: Core Kafka Operations | Author: Rajesh Pasham

Objective: Use the Confluent CLI to produce and consume messages to/from your cluster.

Step-by-Step Instructions

1. Install CLI: brew install confluentinc/tap/confluent (Mac) or download binary
2. Login: confluent login --save
3. Set environment: confluent environment list → note environment ID
4. confluent environment use <env-id>
5. Set cluster: confluent kafka cluster list → confluent kafka cluster use <cluster-id>
6. Create API key: confluent api-key create --resource <cluster-id> --description 'lab-key'
7. Store API key: confluent api-key store <key> <secret> --resource <cluster-id>
8. Produce: confluent kafka topic produce test-events --value-format string
9. Type: {"user": "alice", "action": "login"} and press Enter 5 times
10. Open new terminal. Consume: confluent kafka topic consume test-events --from-beginning

Prerequisites: Exercise 1 completed, CLI installed (brew install confluentinc/tap/confluent)

Validation Criteria

- 5+ messages appear in consumer output
- Consumer shows messages from offset 0
- CLI shows correct cluster endpoint in use
- API key appears in cloud console under API Keys

Pro Tips

- Use --value-format avro in prod with Schema Registry
- Never commit API keys to git — use environment variables
- confluent kafka topic describe test-events shows partition details
- Use --group flag to set consumer group for offset tracking

Hands-On Exercise 3: Set Up a Kafka Connect Source Connector

Topic Area: Data Integration | Author: Rajesh Pasham

Objective: Configure a fully managed HTTP Source Connector to pull data into Confluent Cloud.

Step-by-Step Instructions

1. In Cloud UI, navigate to your cluster → Connectors tab
2. Click 'Add Connector' → search for 'HTTP Source'
3. Select 'HTTP Source (Confluent)' — fully managed connector
4. Configure: API URL = https://jsonplaceholder.typicode.com/posts
5. Set: Topic prefix = api-data, Poll interval = 60000ms
6. Output format: JSON, Auth = None (public endpoint)
7. Click 'Continue' → review config → Click 'Launch'
8. Wait 2 minutes → go to Topics → check api-data-posts topic
9. Use CLI: confluent kafka topic consume api-data-posts --from-beginning
10. Verify JSON records from the placeholder API are flowing in

Prerequisites: Exercise 2 completed, basic understanding of REST APIs

Validation Criteria

- Connector status shows 'Running' in UI
- Topic api-data-posts exists with messages
- Messages contain valid JSON from jsonplaceholder API
- Consumer lag metric shows near-zero (up-to-date)

Pro Tips

- Set sensible poll intervals — too frequent = unnecessary load
- For database CDC connectors, enable exactly-once semantics
- Check 'Dead Letter Queue' topic if connector has errors
- Use SMTs (Single Message Transforms) to mask PII fields

Hands-On Exercise 4: Write ksqlDB Queries for Real-Time Analytics

Topic Area: Stream Processing | Author: Rajesh Pasham

Objective: Use ksqlDB to create streams, filter data, and build real-time aggregations.

Step-by-Step Instructions

1. Go to Cluster UI → ksqlDB tab → Create a ksqlDB application
2. Wait for ksqlDB to provision (1-2 minutes)
3. In the ksqlDB Editor, create a stream:
4. CREATE STREAM posts WITH (kafka_topic='api-data-posts', value_format='JSON');
5. Run a push query to see live data:
6. SELECT userId, title, body FROM posts EMIT CHANGES LIMIT 10;
7. Create a filtered stream for user 1:
8. CREATE STREAM user1_posts AS SELECT * FROM posts WHERE userId = 1;
9. Create an aggregation (count posts per user):
10. CREATE TABLE posts_per_user AS SELECT userId, COUNT(*) AS post_count FROM posts GROUP BY userId;
11. Query the table: SELECT * FROM posts_per_user EMIT CHANGES;

Prerequisites: Exercise 3 completed, topic api-data-posts has data

Validation Criteria

- Stream 'posts' created and shows data
- Filtered stream user1_posts has only userId=1 records
- Table posts_per_user shows count aggregations by user
- Push query emits continuous results as new data arrives

Pro Tips

- Use EMIT CHANGES for streaming (push) vs SELECT for pull queries
- Tables are materialized views — they're stateful and updated continuously
- ksqlDB queries run as Kafka Streams apps under the hood
- Use WINDOW TUMBLING for time-based aggregations (hourly KPIs)

Hands-On Exercise 5: Implement Schema Registry with Avro

Topic Area: Schema Management | Author: Rajesh Pasham

Objective: Register an Avro schema, produce schema-validated messages, and test compatibility.

Step-by-Step Instructions

1. In Cloud UI → Schema Registry → Add schema
2. Choose subject: test-events-value → Format: AVRO
3. Paste schema:
{"type":"record","name":"UserEvent","fields":[{"name":"user_id","type":"int"},{"name":"action","type":"string"},{"name":"timestamp","type":"long"}]}
4. Set compatibility: BACKWARD → Click Save
5. Install Python lib: pip install confluent-kafka[avro]
6. Write producer script using SerializingProducer with AvroSerializer
7. Produce 10 records: {user_id: 1, action: 'login', timestamp: <epoch>}
8. Try adding a new optional field 'device' with default null — re-register schema
9. Verify new schema is version 2 with BACKWARD compatibility check passed
10. Try removing 'action' field — confirm schema registry REJECTS it

Prerequisites: Exercise 2 completed, confluent-kafka Python library or Java Kafka client

Validation Criteria

- Schema registered in Schema Registry UI
- 10 messages produced with Avro serialization
- Version 2 schema (added optional field) accepted
- Breaking schema change (field removal) rejected by registry

Pro Tips

- BACKWARD mode: new schema must be able to read data written by old schema
- Always use default values when adding fields for backward compatibility
- Schema IDs are embedded in every message — no schema duplication
- Use Data Contracts (Stream Governance) to enforce quality rules

Hands-On Exercise 6: Configure RBAC and API Key Security

Topic Area: Security & Access Control | Author: Rajesh Pasham

Objective: Set up Role-Based Access Control, create scoped API keys, and validate access restrictions.

Step-by-Step Instructions

1. Go to cloud.confluent.com → Accounts & access → Service Accounts
2. Create service account: 'producer-svc' with description 'Produces events'
3. Create API key scoped to your cluster for producer-svc
4. In Cluster → RBAC → Role bindings → Add role binding
5. Assign producer-svc the 'DeveloperWrite' role on topic 'test-events'
6. Create another service account: 'consumer-svc' with 'DeveloperRead' on test-events
7. Test: use producer-svc API key to PRODUCE to test-events → should succeed
8. Test: use producer-svc API key to CONSUME from test-events → should FAIL
9. Test: use consumer-svc to CONSUME → should succeed
10. View audit logs: Admin → Audit log topic → check access events

Prerequisites: Confluent Cloud account with Admin role, Exercise 1 completed

Validation Criteria

- Producer service account can only write, not read
- Consumer service account can only read, not write
- Audit log shows all access attempts (success and failure)
- API keys are scoped to specific cluster only

Pro Tips

- Principle of least privilege: grant only what's needed
- Rotate API keys every 90 days — use service accounts not personal keys
- Audit log topic can be exported to SIEM (Splunk, Datadog)
- Use Cloud API Keys for Terraform automation, not cluster-level keys

Hands-On Exercise 7: Set Up Cluster Linking for Disaster Recovery

Topic Area: Migration & Replication | Author: Rajesh Pasham

Objective: Configure Cluster Linking to mirror topics across two clusters for DR.

Step-by-Step Instructions

1. In Confluent Cloud, create a second cluster in a different region (e.g., us-west-2)
2. In the destination cluster, go to Cluster Linking → 'Create link'
3. Select 'Confluent Cloud to Confluent Cloud' as source type
4. Enter source cluster Bootstrap server URL and API credentials
5. Name the link: 'dr-link' → configure auto-create topics: enabled
6. Select topics to mirror: test-events → Launch cluster link
7. Wait 1-2 minutes → verify test-events appears in destination cluster
8. Produce 5 more messages to source cluster's test-events
9. Consume from DESTINATION cluster's test-events → messages should appear
10. Check Cluster Linking metrics: lag should be near 0 (real-time replication)

Prerequisites: Two Confluent Cloud clusters (Basic clusters in different regions work)

Validation Criteria

- test-events topic exists in destination cluster
- All messages from source appear in destination
- Cluster link lag metric < 5 seconds
- Consumer offsets are mapped (consumers can resume from correct offset)

Pro Tips

- Cluster Linking supports both Cloud-to-Cloud and self-managed-to-Cloud
- Use mirror topics for DR — they're read-only in the destination
- For migration, set source-initiated auto-offset-sync to maintain group offsets
- Monitor replication lag with: confluent kafka link describe dr-link

Hands-On Exercise 8: Cost Optimization Lab

Topic Area: Pricing & Cost Management | Author: Rajesh Pasham

Objective: Analyze cost drivers and apply optimization techniques to reduce Confluent Cloud spend.

Step-by-Step Instructions

1. Go to Admin → Billing & payment → View cost breakdown
2. Identify top 3 cost components (compute, storage, egress)
3. Navigate to your cluster → Topics → Sort by Storage
4. Identify topics with high storage but low consumer activity
5. For a topic with 30-day retention: calculate storage cost at $0.08/GB-month
6. Change retention to 7 days: Topics → select topic → Configuration → retention.ms
7. Calculate projected savings from retention change
8. Go to Metrics → check eCKU utilization patterns (when are peaks?)
9. Check if you have idle connectors: Connectors → pause any unused ones
10. Review: Billing estimates → note forecasted monthly cost vs. previous

Prerequisites: Active Confluent Cloud cluster with at least a few days of usage

Validation Criteria

- Identified top cost drivers in billing breakdown
- At least one topic retention period reduced
- Calculated storage savings from retention tuning
- Identified any idle clusters or connectors consuming cost

Pro Tips

- Freight clusters save 60% on storage for logging workloads
- Scale-to-zero: Basic/Standard clusters auto-scale to 0 when idle
- Data transfer costs often > compute — architect to minimize egress
- Use Confluent Cloud Spend Dashboard for trend analysis

Hands-On Exercise 9: AI-Powered Stream Processing with Flink SQL

Topic Area: AI & Real-Time Intelligence | Author: Rajesh Pasham

Objective: Use Flink SQL to create real-time aggregations that simulate ML-based fraud detection logic.

Step-by-Step Instructions

1. Enable Flink in your Confluent Cloud environment (Flink tab in environment)
2. Create a Flink compute pool: 5 CFUs, region matching your cluster
3. In Flink SQL workspace, create table from transactions topic:
4. CREATE TABLE transactions (txn_id STRING, user_id STRING, amount DOUBLE, ts TIMESTAMP(3), WATERMARK FOR ts AS ts - INTERVAL '5' SECOND) WITH (kafka_topic='transactions', value_format='JSON');
5. Produce 20 sample transaction records to 'transactions' topic via CLI
6. Write Flink SQL: detect users with >3 transactions in 10 seconds:
7. SELECT user_id, COUNT(*) AS txn_count, SUM(amount) AS total FROM transactions GROUP BY user_id, TUMBLE(ts, INTERVAL '10' SECOND) HAVING COUNT(*) > 3;
8. Sink high-risk results to a new topic 'fraud-alerts'
9. Consume from fraud-alerts topic and verify high-velocity users are flagged
10. Explore Flink UI: observe job DAG and running operators

Prerequisites: Confluent Cloud account with Flink enabled, test data in a topic

Validation Criteria

- Flink compute pool is running (status: Running)
- Transactions table created successfully from Kafka topic
- Velocity check query identifies users with >3 txns/10sec
- fraud-alerts topic receives flagged records correctly

Pro Tips

- Watermarks are critical for event-time processing — always define them
- TUMBLING windows are non-overlapping; HOPPING windows allow overlap
- Add HAVING clauses AFTER GROUP BY — not in WHERE clause
- For production: add schema validation and DLQ for bad records

KEY TAKEAWAYS

Confluent Cloud in a Nutshell

Speed

Deploy Kafka in minutes vs. months self-managed

Cost

Pay-per-use, scale-to-zero — no waste on idle infra

Ecosystem

Schema Registry, ksqlDB, Flink, 120+ connectors — all managed

Security

SOC 2, ISO 27001, BYOK encryption, Private Link built-in

AI-Ready

Native ML inference, vector search, and AI agent support

Multi-Cloud

Consistent API across AWS, Azure, and GCP